A Two-Stage Building Damage Assessment Pipeline for Post-Earthquake Imagery: Accuracy–Latency Tradeoffs and Cross-Domain Evaluation

Mustafa Eren Soyhan
Student ID: 101045056
Electrical and Computer Engineering, Ontario Tech University
mustafa.soyhan@ontariotechu.net

***Abstract*—This paper develops and evaluates a two-stage building damage assessment pipeline for post-earthquake imagery. The first stage uses YOLOv8-based detectors for building localization, achieving a best mAP@0.5 of 0.528 with YOLOv8-m at 640-pixel resolution. Evaluation at 800-pixel resolution reveals that higher resolution primarily benefits smaller models (YOLOv8-n improves by 1.4%) while larger models show no gain. The second stage employs lightweight CNN classifiers for damage severity prediction, where EfficientNet-B0 attains 90.1% accuracy and 0.443 macro F1-score on the xBD earthquake subset. Systematic benchmarking across three detector sizes, two resolutions, two classifier architectures, and two precision modes (FP32/FP16) reveals that FP16 inference preserves accuracy within 0.3% while latency benefits are hardware-dependent. The best pipeline configuration processes full images at 14.5 FPS with a median latency of 69.2 ms. Cross-domain evaluation on UAVs-TEBDE shows a 35–41% accuracy drop from satellite to UAV imagery, and a supplementary 3-class classifier experiment confirms this gap is driven by visual domain differences rather than label mismatch. These results provide a reproducible evaluation framework that jointly reports detection quality, classification accuracy, runtime performance, and cross-domain degradation patterns.**

***Index Terms*—earthquake damage assessment, building detection, image classification, YOLO, latency benchmarking, cross-domain evaluation**

I. Introduction

Post-earthquake damage assessment is time-critical for emergency response and resource allocation. Rapid and accurate assessment of structural damage enables prioritized deployment of search-and-rescue teams and efficient distribution of humanitarian aid [10]. While deep learning models achieve strong performance on benchmark datasets, practical deployments must also satisfy runtime constraints and maintain robustness under varying imaging conditions. These factors create a gap between research results and deployment-ready systems.

Building damage assessment from imagery involves two distinct sub-tasks: (1) localizing buildings within an image, and (2) classifying the damage severity of each detected building. Many existing approaches address these jointly through semantic segmentation [1], but this coupling makes it difficult to independently evaluate and optimize each component. Furthermore, models trained on satellite imagery may not transfer well to UAV imagery due to differences in viewpoint, resolution, and scene composition [5].

This paper addresses these challenges through a modular two-stage pipeline that separates building localization from damage classification. This design enables independent benchmarking of each stage, facilitates component-level optimization, and provides clearer insight into where performance bottlenecks occur. The paper evaluates accuracy–latency tradeoffs across practical configurations and quantifies cross-domain degradation when transferring from benchmark satellite imagery to UAV imagery collected after real earthquake events.

The contributions are threefold: (1) a modular two-stage pipeline with timing instrumentation enabling reproducible accuracy–latency evaluation; (2) empirical characterization of tradeoffs across model size, input resolution, and inference precision, including the finding that FP16 latency benefits are hardware-dependent and that higher resolution primarily benefits smaller detectors; and (3) cross-domain transfer analysis documenting a 35–41% accuracy drop from satellite to UAV imagery, with a supplementary 3-class experiment confirming the gap is visual rather than label-related.

II. Problem Definition and Scope

Building damage assessment pipelines are typically evaluated on benchmark accuracy alone, while practical deployment requires predictable latency and robustness to domain shift. When models trained on satellite imagery are applied to UAV imagery, differences in perspective and image characteristics can substantially degrade performance. This gap between benchmark evaluation and real-world deployment motivates the need for a more comprehensive assessment framework.

This project investigates three research questions: first, what accuracy–latency tradeoffs exist across model size and input resolution configurations for building localization and damage classification; second, how inference precision (FP32 vs. FP16) affects accuracy and runtime

for each pipeline stage; and third, how much performance degrades when applying satellite-trained models to UAV imagery, and what failure patterns emerge during this domain transfer.

The project uses the earthquake subset of xBD [1] for training and primary evaluation, focusing on earthquake-specific damage patterns rather than all disaster types. Cross-domain evaluation uses the publicly available UAVs-TEBDE dataset [2], which provides UAV imagery from the 2023 Türkiye earthquakes. The implementation uses PyTorch [4] with the Ultralytics YOLO framework [3] for detection and standard classifier architectures for damage classification.

III. Related Work

Satellite-based damage assessment has advanced with large-scale annotated datasets. The xBD dataset [1] provides building annotations and four-level damage labels across 19 disaster events, while Weber and Káné [6] showed that multi-temporal fusion improves damage classification through change detection. Object detection architectures have evolved rapidly, with YOLOv8 [3] introducing anchor-free detection heads that improve performance on densely packed objects. For classification, EfficientNet [7] achieves competitive accuracy through compound scaling, while ResNet [8] provides a well-established transfer learning baseline.

Cross-domain transfer between satellite and UAV imagery remains challenging. Tilon et al. [5] showed that viewpoint differences between nadir satellite and oblique UAV perspectives substantially alter building appearance, requiring adaptation strategies. Mixed-precision inference using FP16 is widely adopted to reduce computation [9], but actual latency benefits depend on hardware architecture, motivating empirical evaluation. Domain-adversarial training [11] offers one approach to bridging domain gaps without requiring large amounts of target-domain labeled data.

Input resolution is a key design parameter for object detection in remote sensing imagery. Higher resolutions preserve fine-grained spatial detail that can improve detection of small and densely packed objects, but at the cost of increased computation and memory requirements. The interaction between model capacity and input resolution is particularly relevant for building detection in satellite imagery, where structures often occupy only a small fraction of the total image area and can be difficult to distinguish from background at lower resolutions. Understanding how resolution interacts with model size enables practitioners to select configurations that balance accuracy against computational budget for specific deployment scenarios.

IV. Methodology

The methodology follows a systematic approach where each pipeline component is independently implemented, trained, and evaluated before integration.

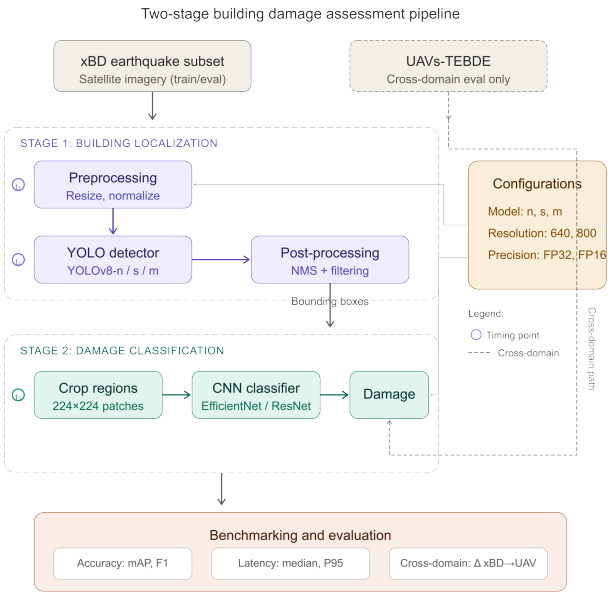

Fig. 1. Two-stage damage assessment pipeline. Stage 1 localizes buildings using YOLO-based detection. Stage 2 classifies damage severity for each detected region using a lightweight CNN classifier. Timing points (t_1, t_2, t_3) indicate where latency is measured. The dashed path shows cross-domain evaluation flow from UAVs-TEBDE.

A. Two-Stage Pipeline Architecture

The pipeline separates building damage assessment into two sequential stages, as illustrated in Figure 1.

The first stage performs building localization using a YOLO-based object detector that identifies building locations in post-disaster imagery, outputting bounding boxes with confidence scores. This stage treats building detection as a single-class problem, deferring damage assessment to the second stage. The detector is trained on building polygons from all disaster types in xBD converted to bounding box format, as building appearance is consistent regardless of disaster type.

The second stage handles damage classification, where a CNN-based classifier predicts damage severity for each detected building region. The classifier receives cropped and resized image patches corresponding to detected buildings and outputs a damage level prediction. This modular design allows the classifier to be trained and evaluated independently of the detector, following established multi-stage approaches for disaster damage assessment [6].

During inference, Stage 1 processes the full image to produce building detections. Each detection above a confidence threshold is cropped from the original high-resolution image (preserving detail), resized to 224×224 pixels, and passed to the Stage 2 classifier. Algorithm 1 formalizes the inference procedure with timing instrumentation.

Algorithm 1 Two-Stage Inference with Timing

Require: Image I, detector D, classifier C, thresholds τ_{conf}, τ_{nms}

Ensure: Detections with damage labels, timing metrics

$t_0 \leftarrow \text{TIME}()$
$I' \leftarrow \text{PREPROCESS}(I)$
$t_1 \leftarrow \text{TIME}()$
$boxes \leftarrow D(I')$ ▷ Stage 1: Detection
$boxes \leftarrow \text{NMS}(boxes, \tau_{conf}, \tau_{nms})$
$t_2 \leftarrow \text{TIME}()$
$results \leftarrow \emptyset$
for each $box \in boxes$ **do**
  $patch \leftarrow \text{CROP}(I, box)$ ▷ From original image
  $damage \leftarrow C(patch)$ ▷ Stage 2: Classify
  $results \leftarrow results \cup \{(box, damage)\}$
end for
$t_3 \leftarrow \text{TIME}()$
return $results$, $(t_1 - t_0, t_2 - t_1, t_3 - t_2)$

B. Dataset Preparation

The xBD dataset contains pre- and post-disaster satellite imagery (1024×1024 pixels) across 19 disaster events with building polygon annotations in WKT format and four-level damage labels [1]. For detector training, all disaster types are used, yielding 2,283 post-disaster images containing 162,787 building bounding boxes, split 80/20 into 1,831 training and 452 validation images stratified by disaster type.

For classifier training, only earthquake-related events are used (mexico-earthquake and palu-tsunami prefixes) to maintain domain consistency. This produces 63,186 building patches with a highly imbalanced class distribution: no-damage constitutes 91.0%, destroyed 7.9%, major-damage 0.9%, and minor-damage only 0.2% (92 training samples). This imbalance is addressed through inverse-frequency class weighting in the loss function.

The UAVs-TEBDE dataset [2] provides pre-cropped UAV building patches from the 2023 Türkiye earthquakes with three categories: Intact (498), Damaged (664), and Collapsed (474). For cross-domain evaluation, xBD labels are harmonized to three classes: *no-damage*→*Intact*, *minor-damage* and *major-damage*→*Damaged*, and *destroyed*→*Collapsed*. This harmonization is applied during evaluation only.

C. Experimental Configurations

Table I summarizes the configuration space. Three YOLOv8 model sizes (nano: 3M parameters, small: 11M, medium: 25M) represent different capacity–speed trade-offs, evaluated at both 640 and 800 pixel input resolutions. For classification, EfficientNet-B0 [7] and ResNet-34 [8] are evaluated using 224×224 input patches. All configurations are tested in both FP32 and FP16 precision modes.

TABLE I
EXPERIMENTAL CONFIGURATION SPACE

Component	Parameter	Values
Detector	Model size	YOLOv8-n, -s, -m
	Resolution	640, 800
Classifier	Architecture	EfficientNet-B0, ResNet-34
	Input size	224×224
Both	Precision	FP32, FP16

D. Training Protocol

YOLO detectors are trained for up to 100 epochs with early stopping (patience=15) based on validation mAP. Standard augmentation includes mosaic, mixup, and geometric transforms, with batch sizes adjusted per model to fit within 12 GB VRAM (YOLOv8-n: 16/8, YOLOv8-s: 8/4, YOLOv8-m: 4/2 for 640/800 resolution respectively). The Ultralytics framework automatically selects the AdamW optimizer with learning rate 0.002.

Classifiers are initialized with ImageNet-pretrained weights and fine-tuned for 50 epochs using AdamW (lr = 10^{-3}, weight decay= 10^{-4}) with cross-entropy loss weighted by inverse class frequency. The learning rate schedule consists of a 5-epoch linear warm-up followed by cosine decay. Data augmentation includes random horizontal and vertical flips, rotation (±15°), and color jitter. All experiments use a fixed random seed (42) with deterministic CUDA operations for reproducibility.

E. Evaluation Protocols

Detector performance is measured using mAP@0.5 and mAP@0.5:0.95. Classifier performance is measured using overall accuracy and macro-averaged F1-score, which accounts for class imbalance by weighting each class equally regardless of sample count.

The latency protocol includes 10 warm-up inference passes, evaluation on 50 validation images with `torch.cuda.synchronize()` for accurate GPU timing, and reporting of both median and 95th-percentile (P95) latency. Stage-level timing separately measures preprocessing, detection, and classification. Throughput is reported as frames per second (FPS).

For cross-domain evaluation, trained classifiers are applied directly to UAVs-TEBDE without fine-tuning, using the label harmonization described above. Additionally, we train a 3-class classifier (merging minor and major damage into "Damaged") to test whether the label harmonization during evaluation contributes to the observed domain gap. This evaluation is explicitly exploratory: we document the magnitude and nature of domain shift rather than expecting strong transfer performance.

F. Experimental Setup

All experiments are implemented in Python using PyTorch 2.5.1 with CUDA 12.x [4], with detection models

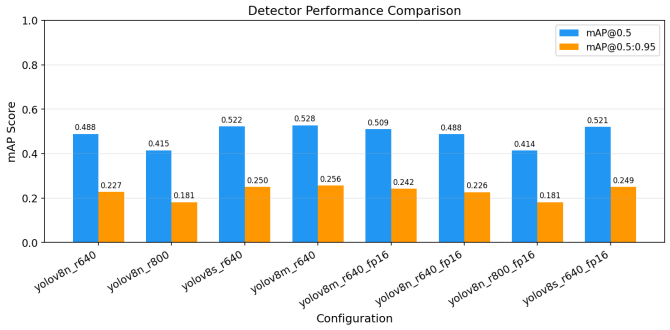

Fig. 2. Detector performance comparison across all YOLOv8 configurations showing mAP@0.5 and mAP@0.5:0.95. FP32 and FP16 results are shown side by side for each model size and resolution.

from Ultralytics YOLOv8 [3]. The hardware platform consists of an NVIDIA RTX 4080 Laptop GPU (12 GB VRAM), 32 GB system RAM, and an Intel Core i9-13900HX processor running Windows. All code, trained weights, and evaluation results are released for reproducibility.

V. Results

A. Detector Performance

Table II presents detection results across all configurations at both resolutions. YOLOv8-m at 640 achieves the highest mAP@0.5 of 0.528, followed by YOLOv8-s at 800 with 0.525 and YOLOv8-s at 640 with 0.522. The consistent improvement from nano to medium, visible in Figure 2, confirms that increased model capacity benefits building localization on dense satellite imagery with many small, closely spaced buildings.

Higher resolution (800) primarily benefits smaller models: YOLOv8-n improves from 0.488 to 0.502 (+1.4%), while YOLOv8-s shows a marginal gain from 0.522 to 0.525 (+0.3%), and YOLOv8-m shows no benefit (0.528→0.525). This suggests that larger models already capture sufficient spatial detail at 640 resolution, while smaller models benefit from the additional pixel information at 800. Notably, YOLOv8-m at 800 achieves the highest precision (0.704) across all configurations, indicating that higher resolution improves localization quality even when overall mAP does not increase.

FP16 inference has negligible impact on YOLOv8-n and YOLOv8-s (<0.1% mAP@0.5 difference), but produces a more notable 1.9% drop for YOLOv8-m, suggesting that larger models are more sensitive to quantization effects during inference.

B. Classifier Performance

Table III shows classifier results. EfficientNet-B0 achieves 90.1% accuracy and 0.443 macro F1, outperforming ResNet-34 (84.8% accuracy, 0.413 macro F1) across both precision modes. FP16 inference has virtually no impact on either architecture (<0.1% difference).

TABLE II
Detector Performance Across Resolutions and Precision

Config	mAP50	mAP50-95	Prec.	Recall
YOLOv8-n r640 FP32	0.488	0.227	0.664	0.445
YOLOv8-n r640 FP16	0.488	0.226	0.661	0.445
YOLOv8-n r800 FP32	0.502	0.240	0.682	0.454
YOLOv8-s r640 FP32	0.522	0.250	0.682	0.481
YOLOv8-s r640 FP16	0.521	0.249	0.680	0.480
YOLOv8-s r800 FP32	0.525	0.257	0.695	0.478
YOLOv8-m r640 FP32	**0.528**	0.256	0.684	**0.486**
YOLOv8-m r640 FP16	0.509	0.242	0.682	0.462
YOLOv8-m r800 FP32	0.525	**0.257**	**0.704**	0.471

TABLE III
Classifier Performance (FP32 and FP16 Inference)

Config	Accuracy	Macro F1
EfficientNet-B0 FP32	**0.901**	**0.443**
EfficientNet-B0 FP16	0.901	0.442
ResNet-34 FP32	0.848	0.413
ResNet-34 FP16	0.849	0.413

The relatively low macro F1 despite high accuracy reflects the severe class imbalance. Figure 3 shows the per-class breakdown for EfficientNet-B0: both classifiers perform well on no-damage (F1>0.91) and destroyed (F1: 0.60–0.68), but minor-damage achieves near-zero recall due to having only 92 training samples. Major-damage (F1: 0.12–0.14) also suffers from scarcity. Despite inverse-frequency weighting, the model cannot reliably learn discriminative features from so few examples of intermediate damage levels.

The confusion matrix in Figure 4 provides further insight into error patterns. EfficientNet-B0 correctly classifies 92% of no-damage and 77% of destroyed buildings, but misclassifies 52% of major-damage as no-damage and 100% of minor-damage as no-damage. This confirms that the classifier defaults to the dominant class when uncertain, a characteristic failure mode under extreme imbalance.

C. Latency Benchmarking

Table IV presents latency results for end-to-end pipeline configurations using the best detector (YOLOv8-s) paired with each classifier. The fastest pipeline is YOLOv8-s + ResNet-34 in FP32 mode, achieving 14.5 FPS with 69.2 ms median latency. ResNet-34 yields a faster pipeline than EfficientNet-B0 despite lower standalone accuracy, due to its simpler architecture requiring fewer operations per forward pass.

The high P95 latency (up to 629 ms) reflects variance in the number of detected buildings per image: images with many detections require proportionally more classifier forward passes, creating a long tail in the latency distribution.

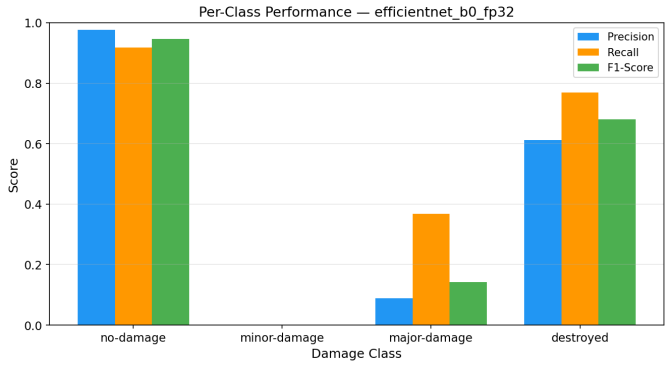

Fig. 3. Per-class precision, recall, and F1-score for EfficientNet-B0 (FP32). The model achieves strong performance on no-damage and destroyed classes but near-zero scores for minor-damage due to extreme class imbalance (only 92 training samples).

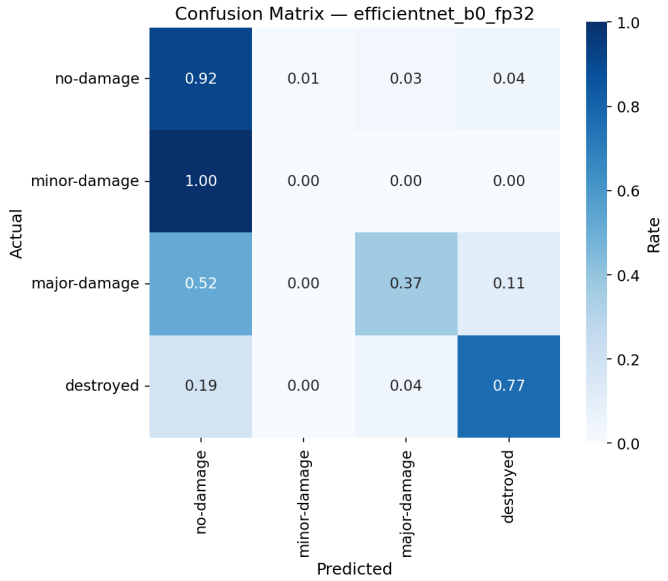

Fig. 4. Confusion matrix for EfficientNet-B0 (FP32) on xBD validation. The model strongly classifies no-damage (92%) and destroyed (77%) but defaults to no-damage for intermediate damage levels due to extreme class imbalance.

A notable finding is that FP16 does not reduce latency on this hardware. All FP16 pipeline configurations are slower than their FP32 counterparts, likely due to overhead from FP16↔FP32 type conversions on the RTX 4080 Laptop GPU architecture. This contradicts the common assumption that reduced precision always provides speedup [9].

D. Cross-Domain Evaluation

Table V presents cross-domain transfer results from xBD satellite imagery to UAVs-TEBDE UAV imagery. Both classifiers experience substantial accuracy drops: EfficientNet-B0 drops from 90.1% to 49.0% ($\Delta = -41.1\%$), while ResNet-34 drops from 84.8% to 49.3% ($\Delta = -35.6\%$).

TABLE IV
End-to-End Pipeline Latency (YOLOv8-s Detector)

Pipeline Configuration	Median	P95	FPS
+ ResNet-34 FP32	**69.2 ms**	**290.8 ms**	**14.5**
+ ResNet-34 FP16	81.3 ms	381.1 ms	12.3
+ EfficientNet-B0 FP32	83.5 ms	419.4 ms	12.0
+ EfficientNet-B0 FP16	128.4 ms	628.7 ms	7.8

TABLE V
Cross-Domain Transfer: xBD → UAVs-TEBDE

Classifier	xBD Acc.	TEBDE Acc.	Δ
EfficientNet-B0	0.901	0.490	−0.411
ResNet-34	0.848	0.493	−0.356

Confusion matrix analysis (Figure 5) reveals systematic failure patterns. EfficientNet-B0 achieves per-class accuracy of 33% for Intact, 79% for Damaged, and only 24% for Collapsed on TEBDE. The model exhibits a strong bias toward predicting "Damaged": 67% of Intact buildings and 68% of Collapsed buildings are misclassified as Damaged. ResNet-34 shows a similar but more extreme pattern, with 40% Intact accuracy, 86% Damaged accuracy, and only 7% Collapsed recall.

Interestingly, ResNet-34 shows a smaller absolute delta (−35.6% vs. −41.1%) despite lower xBD accuracy, suggesting that simpler architectures may be marginally more robust to domain shift due to less overfitting to source-domain features.

E. 3-Class Classifier Experiment

To determine whether the cross-domain gap is caused by label mismatch during harmonization or by fundamental visual differences, we trained 3-class classifiers that directly output Intact, Damaged, and Collapsed—matching the TEBDE label scheme exactly. During training, xBD *minor-damage* and *major-damage* are merged into a single *Damaged* class, pooling 575 training samples instead of splitting them across two rare categories. Table VI presents the comparison.

Training directly on three classes did not improve cross-domain transfer—TEBDE accuracy is slightly worse for both architectures (EfficientNet-B0: 49.0%→41.0%, ResNet-34: 49.3%→46.6%). This is a valuable negative result: it confirms that the domain gap is driven by visual feature differences between satellite and UAV imagery (viewpoint, resolution, scene composition) rather than by the label harmonization scheme used during evaluation. The classifiers learn satellite-specific features such as rooftop debris patterns visible from nadir view that do not transfer to the oblique UAV perspective where damage manifests as wall cracks, floor pancaking, and structural leaning.

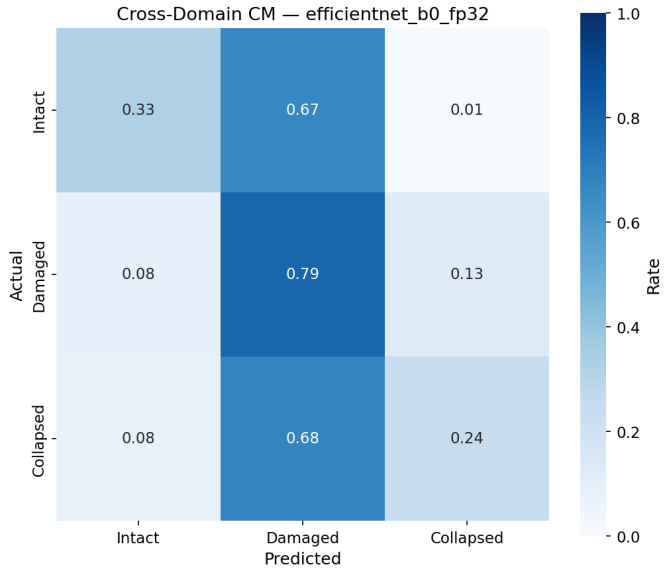

Fig. 5. Cross-domain confusion matrix for EfficientNet-B0 on UAVs-TEBDE. The model shows strong bias toward predicting "Damaged," misclassifying the majority of Intact and Collapsed buildings into this category.

TABLE VI
3-Class vs 4-Class Cross-Domain Comparison

Classifier	Scheme	xBD	TEBDE	Δ
EfficientNet-B0	4-class	0.901	0.490	−0.411
	3-class	0.900	0.410	−0.491
ResNet-34	4-class	0.848	0.493	−0.356
	3-class	0.880	0.466	−0.414

VI. Discussion

A. Resolution and Model Size Tradeoffs

The 800-pixel resolution results reveal an interaction between model capacity and input resolution. Smaller models benefit more from higher resolution: YOLOv8-n improves by 1.4% at 800 vs. 640, while YOLOv8-m shows no improvement. This suggests that larger models already capture sufficient spatial detail at 640 resolution through their greater representational capacity, while smaller models lack this capacity and benefit from additional pixel information. For practitioners, this means 800 resolution is worthwhile primarily for lightweight deployment scenarios using smaller detectors, while larger models can operate at 640 without accuracy loss.

B. Accuracy–Latency Tradeoffs

The detector results show a clear capacity–accuracy tradeoff: YOLOv8-m achieves the highest mAP but YOLOv8-s reaches 98.9% of this performance with substantially fewer parameters (11M vs. 25M). For the full pipeline, the classifier choice has a larger impact on end-to-end latency than the detector choice, since multiple classifier forward passes are required per image—one for each detected building. This makes the per-patch inference cost of the classifier the dominant factor in overall throughput.

The practical implication is that for deployment scenarios where multiple buildings per image are expected, optimizing classifier inference speed yields greater throughput improvements than optimizing the detector.

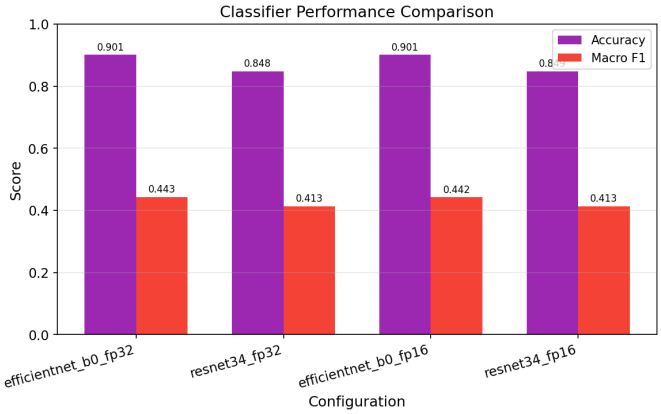

Fig. 6. Classifier accuracy and macro F1 across all configurations. FP16 inference produces virtually identical results to FP32 for both architectures, confirming that reduced precision does not degrade classification quality.

C. FP16 Precision Effects

FP16 inference consistently preserves accuracy across all configurations (within 0.3% for detectors, <0.1% for classifiers), as visualized in Figure 6. This confirms that reduced-precision inference is viable for this application without meaningful quality loss. However, the expected latency reduction did not materialize on the RTX 4080 Laptop GPU: FP16 configurations were consistently slower than FP32. This finding is important for practitioners who may assume FP16 always provides speedup. The actual benefit depends on GPU architecture, model structure, and driver-level optimization for mixed-precision operations [9].

D. Class Imbalance

The severe class imbalance in the xBD earthquake subset (91% no-damage, 0.2% minor-damage) fundamentally limits macro F1 performance. With only 92 training examples for minor-damage, even aggressive class weighting cannot compensate for insufficient sample diversity. Alternative strategies such as minority oversampling, synthetic augmentation, or merging intermediate damage levels into a single "damaged" category could alleviate this limitation.

E. Cross-Domain Transfer

The 35–41% accuracy drop on TEBDE underscores a fundamental challenge: satellite-trained models do not generalize to UAV imagery without adaptation. The systematic "Damaged" bias reveals that the classifiers have

learned satellite-specific features (e.g., rooftop debris patterns visible from nadir view) that manifest differently in oblique UAV imagery, where facades and structural deformation are more prominent. The 3-class experiment further strengthens this conclusion by ruling out label harmonization as a contributing factor—even when the model trains directly on the same three classes used by TEBDE, performance does not improve.

The practical implication is significant: deploying satellite-trained models on UAV imagery without domain adaptation would produce unreliable assessments, particularly for collapsed structures where only 7–24% are correctly identified—precisely the category most critical for emergency response. Domain adaptation techniques such as fine-tuning on a small labeled subset or domain-adversarial training [11] represent promising directions.

VII. Future Work

Several directions could extend this work. Oversampling minority classes through weighted random sampling or applying synthetic augmentation could address data scarcity for rare damage levels and improve macro F1 performance. For cross-domain transfer, domain adaptation strategies such as fine-tuning on a small TEBDE subset or adversarial domain adaptation [11] could reduce the satellite-to-UAV performance gap. Finally, comparing the modular two-stage pipeline against end-to-end joint training approaches would clarify whether the flexibility of modularity outweighs the optimization advantages of joint learning.

VIII. Conclusion

This paper presented a modular two-stage pipeline for post-earthquake building damage assessment with comprehensive benchmarking across model configurations, resolutions, precision modes, and imaging domains. The best detector (YOLOv8-m at 640×640) achieves 0.528 mAP@0.5, while evaluation at 800×800 resolution shows that higher resolution primarily benefits smaller models (+1.4% for YOLOv8-n) with no gain for larger models. EfficientNet-B0 achieves 90.1% classification accuracy with 0.443 macro F1. The optimal end-to-end pipeline (YOLOv8-s + ResNet-34, FP32) processes images at 14.5 FPS with 69.2 ms median latency. FP16 inference preserves accuracy but does not consistently improve latency on laptop GPU hardware, highlighting the importance of empirical benchmarking over assumed speedup. Cross-domain evaluation reveals a 35–41% accuracy drop from satellite to UAV imagery, and a supplementary 3-class experiment confirms this gap is driven by visual domain differences rather than label mismatch. The complete implementation, trained models, and evaluation framework are publicly released for reproducibility.

References

[1] R. Gupta, B. Goodman, N. Patel, R. Hosfelt, S. Sajeev, E. Heim, J. Doshi, K. Lucas, H. Choset, and M. Gaston, "Creating xBD: A dataset for assessing building damage from satellite imagery," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019, pp. 10–17.

[2] M. N. Mowla and D. Asadi, "UAVs-based Turkey Earthquake Building Damage Estimation Dataset (UAVs-TEBDE)," Mendeley Data, V1, 2025. [Online]. Available: https://doi.org/10.17632/5m349hfvkb.1

[3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[4] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[5] S. Tilon, F. Nex, N. Kerle, and G. Vosselman, "Post-disaster building damage detection from earth observation imagery using unsupervised and transferable anomaly detecting generative adversarial networks," *Remote Sens.*, vol. 12, no. 24, p. 4193, 2020.

[6] E. Weber and H. Káné, "Building disaster damage assessment in satellite imagery with multi-temporal fusion," in *Proc. IEEE/CVF Int. Conf. Computer Vision Workshops (ICCVW)*, 2020, pp. 4571–4580.

[7] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Machine Learning (ICML)*, 2019, pp. 6105–6114.

[8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[9] P. Micikevicius *et al.*, "Mixed precision training," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.

[10] L. Dong and J. Shan, "A comprehensive review of earthquake-induced building damage detection with remote sensing techniques," *ISPRS J. Photogramm. Remote Sens.*, vol. 84, pp. 85–99, 2013.

[11] Y. Ganin *et al.*, "Domain-adversarial training of neural networks," *J. Machine Learning Research*, vol. 17, no. 59, pp. 1–35, 2016.